

96GBのPCで DeepSeek V4 Flashを動かしたい！

大きなモデルを、手元の1台で。「動いた」けれど「結局遅い」？ エージェント実用へのリアルな現在地。

モデル：DeepSeek V4 Flash
商用モデルとも比較される巨大モデル。

PC：Strix Halo・96 GiB
Ryzen AI MAX+ 395 / Radeon 8060S。

最適化の軌跡 | 試行ごとの生成速度

01 / まず動かす

約12 tok/s

96GBに収めて、手元で動かす

02 / 実装を変える

約17 tok/s

このPC向けの実装に切り替え

03 / 高速化処理を試す

約20 tok/s

「先読み」による高速化を追加

検証の結論 入った！ 速くなった（約21 tok/s）！ でもエージェント常用には「結局遅い」

そもそも、96GBに入るのか？

「96GBあれば余裕」と思いきや、実はギリギリでした。
CPUとGPUで共有するため、96GB丸ごとをモデルに使えるわけではない。

モデルだけで約89%

OS認識RAM 91 GiB を基準



重みのファイルサイズ

残り：OS・実行環境 + KVキャッシュ

モデルは入った。
でも、残りはたった数GB。

モデルだけで約81GB。OS分を引くと残りは約10GB未満しかなく、この狭い隙間で「長い会話」と「高速化」を両立できるかが最初の壁となった。

それでも載せたい理由 - Claude Opus 4.8とのベンチマークの比較		
Terminal Bench 2.1	DeepSWE	NL2Repo
82.7 / 85.0	54.4 / 58.0	54.2 / 69.7

圧縮したモデルを、96GBで動かす

本来なら到底動かない超巨大モデルを、実用性を損なわない限界まで圧縮し、このパソコンで動かした。

最初の達成 約81GBまで圧縮して読み込みに成功。実際に文章を生成できた。

モデル / WEIGHTS

まずは圧縮してねじ込む

賢さを維持できるギリギリまでデータを圧縮し、**96GBの枠内に収めた**。

実行 / INFERENCE

手元でちゃんと動いた！

専用ソフトで動かして**短い対話を確認**。Web系SWE-bench 30問では、Q2版もクラウドのフル精度版も**20/30 (66.7%)**を解決し、総合スコアの品質劣化は見られなかった。

ここでの到達点 起動には成功したものの、メモリの余白はわずか。次は「使われない部分を削る」手法を試す。

モデルを削れば、もっと余裕ができる？

全体の圧縮に続いて、普段あまり使われない部分を切り落とす手法（REAP）を試した。

試したこと

出番の少ない部分を削る

利用頻度の低い回路を削ってモデルをスリム化し、浮いたメモリを会話や高速化に回そうと考えた。



結果

まさかの日本語崩壊…

容量は削減できたものの、日本語で入力すると怪文書が返る状態に。普段使いのモデルとしては採用を見送った。

アシスタント

Віс. На рус. Привет. Можно предложить help.

ユーザー

こんにちは

この試行の結論 「読み込める」 だけでは不十分。自然な日本語で会話できることが必須条件。

動かす方法を変えて、生成を速くする

96GBに入っても、文字が出るのをじっと待つようでは使いづらい。
次の課題は、生成スピードだった。

mainline llama.cpp / Vulkan

約11 tok/s



strix-halo-llamacpp / Vulkan

約17 tok/s

動くけれど、待たされる

DwarfStarの初期構成では文字が出るテンポが遅く、実用には物足りない。メモリに収めるだけでは快適に使えないことがわかった。

このパソコン専用の実装に乗り換え

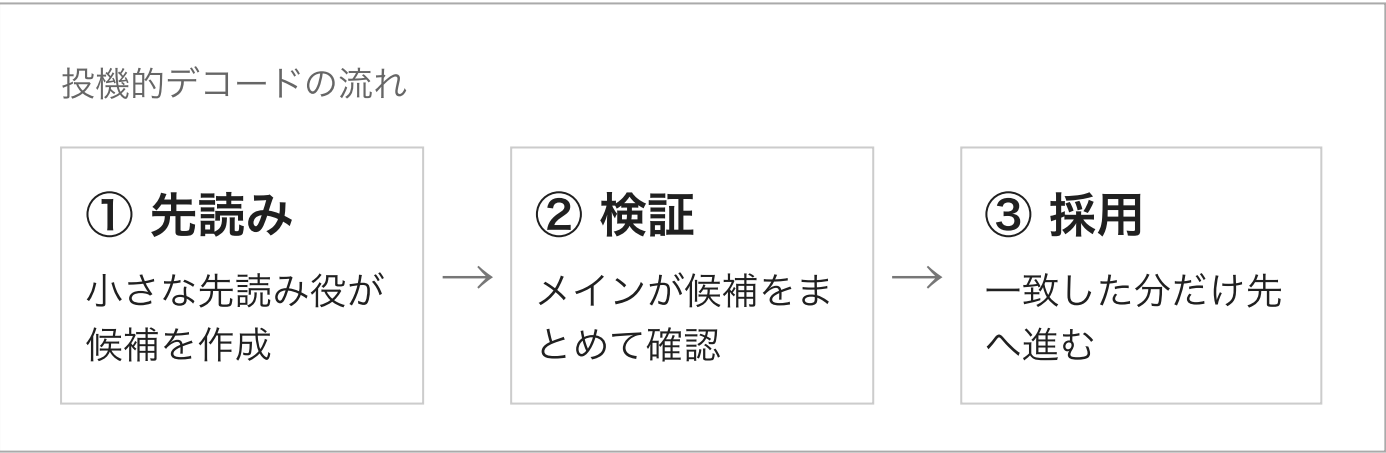
既存の高速化手法（ds4fa）ではメモリ不足で断念。そこで他の、このPC専用に最適化された**strix-halo-llamacpp**へ切り替え、速度を上げることができた。

高速化手法を取り入れてみる

さらに速く生成したい。ただ「先読み役」を追加すると、今度はメモリが足りない。

「先読み」でまとめて文字を出す

毎回大きなメインモデルを動かす代わりに、軽い「先読み役」に次の単語候補を複数作らせ、メインがまとめて検証する仕組みを導入した。



先読み役を載せると、メモリ不足に

すでにギリギリのメモリにもう1つモデルを載せるため、長い会話を試すとメモリ不足で止まってしまう問題が生じた。

速くするために追加した先読み役自体も、メモリを圧迫しないよう軽量化する必要に迫られた。

本体を削ると日本語が崩れる。それなら「先読み役」だけを削る。

4枚目の失敗を踏まえ、メインではなく先読み役だけを軽量化。約2GBのメモリを節約しつつ、生成スピードもほぼ維持できた。

で、実際どれくらい速い？

先読みで生成スピードが約1.3倍に。
先読み役を軽量化しても、速さはほぼ同じ。

出力する速さ Decode

条件を揃えて測定した、1秒あたりの生成トークン数（各構成8回の平均）



入力を読む速さ Prefill

1,233トークンの指示を入力したときの処理速度を比較



先読みはどれくらい当たる？ 採用率

先読み役が出した候補のうち、メインモデルに採用された割合



結局、実用レベルになった？

96GBの制約でも動かすことはできた。速くもなった。 ただ、エージェント用途ではまだ遅い...

約21 tok/sまで引き上げたものの、常用には30~50 tok/s欲しい。さらに長文ファイル読込（Prefill）の遅さやスワップの懸念も残った。

使用感 | 生成速度（Decode）

回答に数分待ち。最低30 tok/sは必要

普段のエージェント利用では50 tok/s超を使っており、最低でも30 tok/sが実用ライン。約21 tok/sでは回答に数分待たされ厳しい。

使用感 | 読込（Prefill）とメモリ

ファイル読込の遅さと、スワップの影

コードやファイルを読み込むPrefillに時間がかかる。また、検証中にわずかなスワップ使用履歴も確認された。96GBあっても極限状態のため、他アプリを極力落とした専用環境での運用が前提になる。

今後の展望

エージェント実用の目安となる30 tok/s 台を目指す。ファイル読込（Prefill）の高速化やさらなる軽量化を進め、これ以上メモリ使用量を増やさない範囲で最適運用を追求したい。



noteで続きの研究を掲載予定！

展示の深掘り・やり残した最適化の検証

展示で触れきれなかった詳細情報やログに加え、カーネル最適化など今回やり足りなかった技術的挑戦の続きをnoteで順次公開予定！

<https://note.com/observer2003>